

Data Structures



Lecture 2 Multidimensional Array

Prantik Paul [PNP]

Lecturer

Department of Computer Science and Engineering
BRAC University

MultiDimensional Array (Initialize)

1D Array

```
array1D = np.zeros(5)
```

2D Array

```
array2D = np.zeros((5,2))
```

3D Array

```
array3D = np.zeros(?)
```

MultiDimensional Array (Initialize)

1D Array

```
array1D = np.zeros(5, dtype = int)
```

2D Array

```
array2D = np.zeros((5,2), dtype = int)
```

3D Array

```
array3D = np.zeros(?)
```

MultiDimensional Array (Initialize)

1D Array

```
array1D = np.array([1,2])
```

2D Array

```
array2D = np.array([[1,2], [3,4]])
```

3D Array

```
array3D = np.array(?)
```

MultiDimensional Array (Initialize)

1D Array

```
array1D = np.array([1,2])
```

2D Array

```
array2D = np.array([[1,2], [3,4]])
```

3D Array

```
array3D = np.array([[[1,2],[3,4]],[[5,6],[7,8]])
```

MultiDimensional Array (In Memory)

Row Major Ordering

1	2	3
4	5	6
7	8	9



row-major

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

MultiDimensional Array (In Memory)

Column Major Ordering

1	2	3
4	5	6
7	8	9

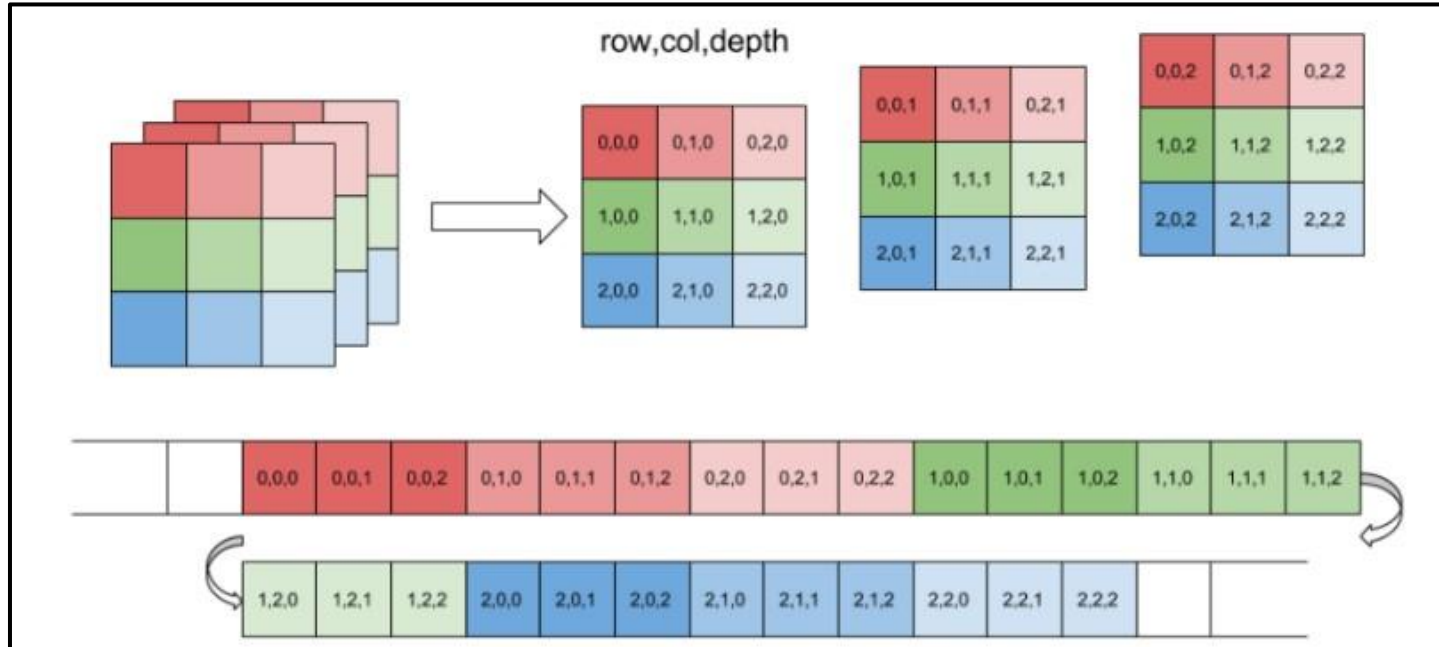


column-major

1	4	7	2	5	8	3	6	9
---	---	---	---	---	---	---	---	---

MultiDimensional Array (In Memory)

3D Array



MultiDimensional Array (Index Finding)

3D Array

```
arr[M][N][O]
```

MultiDimensional Array (Index Finding)

4D Array

```
arr[M][N][O][P]
```

MultiDimensional Array (Reverse Indexing)

3D Array

arr[M][N][O]

M = 4

N = 4

O = 8

Location 111

MultiDimensional Array (Reverse Indexing)

3D Array

$$111 = M * (4*8) + N * 8 + O$$

$$X = 111 // (4*8) = 3 \text{ and } 111 \% (4*8) = 15$$

$$Y = 15 // 8 = 1 \text{ and } 15 \% 8 = 7$$

$$Z = 7$$

MultiDimensional Array (Reverse Indexing)

3D Array

96, 107, 60

MultiDimensional Array (Iteration - array)

2D Array

MultiDimensional Array (Iteration - array)

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    print(x)
```

MultiDimensional Array (Iteration - element)

2D Array

MultiDimensional Array (Iteration - element)

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    for y in x:
        print(y)
```

MultiDimensional Array (Iteration - element)

3D Array

MultiDimensional Array (Iteration - element)

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
                [[7, 8, 9], [10, 11, 12]]])

for x in arr:
    for y in x:
        for z in y:
            print(z)
```

2D Array (Shape)

```
m = np.zeros((2,3), dtype=int)  
print(m.shape)
```

```
(2, 3)
```

2D Array (Iteration - Row Wise)

```
FUNCTION print_row(m: ARRAY)
  DECLARE row, col: INTEGER
  row, col = m.DIMENSIONS
  FOR i = 0 TO row-1
    FOR j = 0 TO col-1
      PRINT m[i][j] + " "
    END FOR
    PRINT "\n"
  END FOR
END FUNCTION
```

2D Array (Iteration - Row Wise)

```
def print_row(m):  
    row, col = m.shape  
    for i in range(row):  
        for j in range(col):  
            print(m[i][j], end = ' ' )  
        print()
```

2D Array (Iteration - Column Wise)

```
FUNCTION print_col(m: ARRAY)
  DECLARE row, col: INTEGER
  row, col = m.DIMENSIONS
  FOR i = 0 TO col-1
    FOR j = 0 TO row-1
      PRINT m[j][i] + " "
    END FOR
    PRINT "\n"
  END FOR
END FUNCTION
```

2D Array (Iteration - Column Wise)

```
def print_col(m):  
    row, col = m.shape  
    for i in range(col):  
        for j in range(row):  
            print(m[j][i], end = ' ' )  
        print()
```

2D Array (Summation)

```
FUNCTION array_sum(m: ARRAY)
  DECLARE sum, row, col: INTEGER
  sum = 0
  row, col = m.DIMENSIONS
  FOR i = 0 TO row-1
    FOR j = 0 TO col-1
      sum = sum + m[i][j]
    END FOR
  END FOR
  RETURN sum
END FUNCTION
```

2D Array (Summation)

```
def array_sum(m):  
    sum = 0  
    row, col = m.shape  
    for i in range(row):  
        for j in range(col):  
            sum += m[i][j]  
    return sum
```

2D Array (Summation along Rows)

4	3	8	→	15
2	5	1	→	8
7	-1	9	→	9
5	4	-2	→	7

2D Array (Summation along Rows)

```
FUNCTION row_wise_sum(m: ARRAY)
  DECLARE row, col: INTEGER
  DECLARE result: ARRAY [0:row-1, 0:0] OF INTEGER
  row, col = m.DIMENSIONS
  FOR i = 0 TO row-1
    FOR j = 0 TO col-1
      result[i][0] = result[i][0] + m[i][j]
    END FOR
  END FOR
  RETURN result
END FUNCTION
```

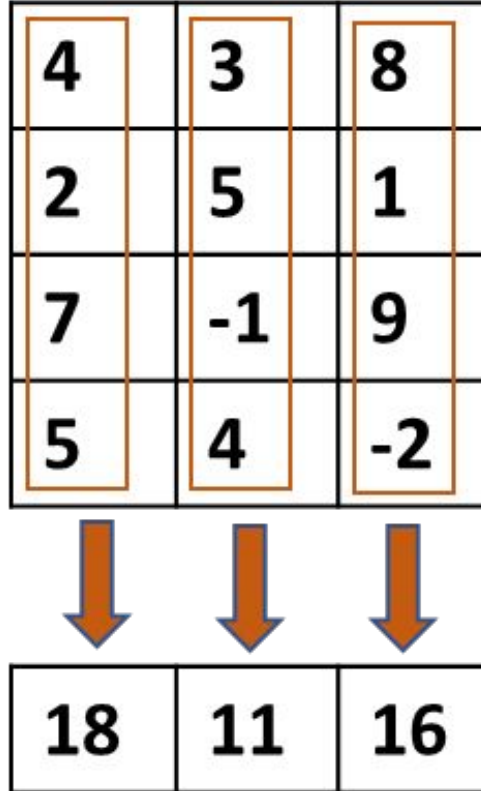
2D Array (Summation along Rows)

```
def row_wise_sum(m):  
    row, col = m.shape  
    result = np.zeros((row,1), dtype = int)  
    for i in range(row):  
        for j in range(col):  
            result[i][0] += m[i][j]  
    return result
```

2D Array (Summation along Columns)

4	3	8
2	5	1
7	-1	9
5	4	-2

18	11	16
----	----	----



The diagram illustrates the process of summing elements along the columns of a 2D array. The top array is a 4x3 grid with values: Row 1 [4, 3, 8], Row 2 [2, 5, 1], Row 3 [7, -1, 9], and Row 4 [5, 4, -2]. Three orange arrows point downwards from each column to a single-row array below. The bottom array contains the column sums: 18 (4+2+7+5), 11 (3+5+(-1)+4), and 16 (8+1+9+(-2)).

2D Array

Find The Largest Number in a 2D Array

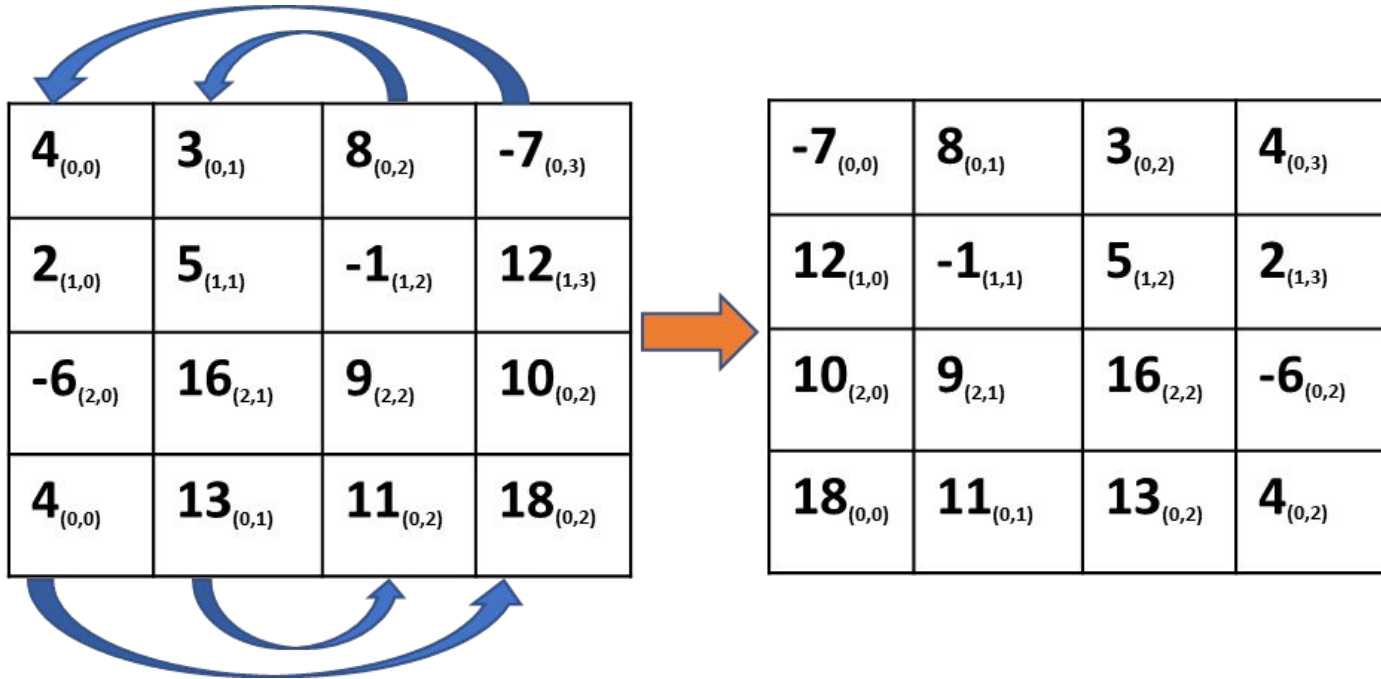
2D Array

Find The Largest Number in Each Row in a 2D Array. Then Find The Smallest Among Them

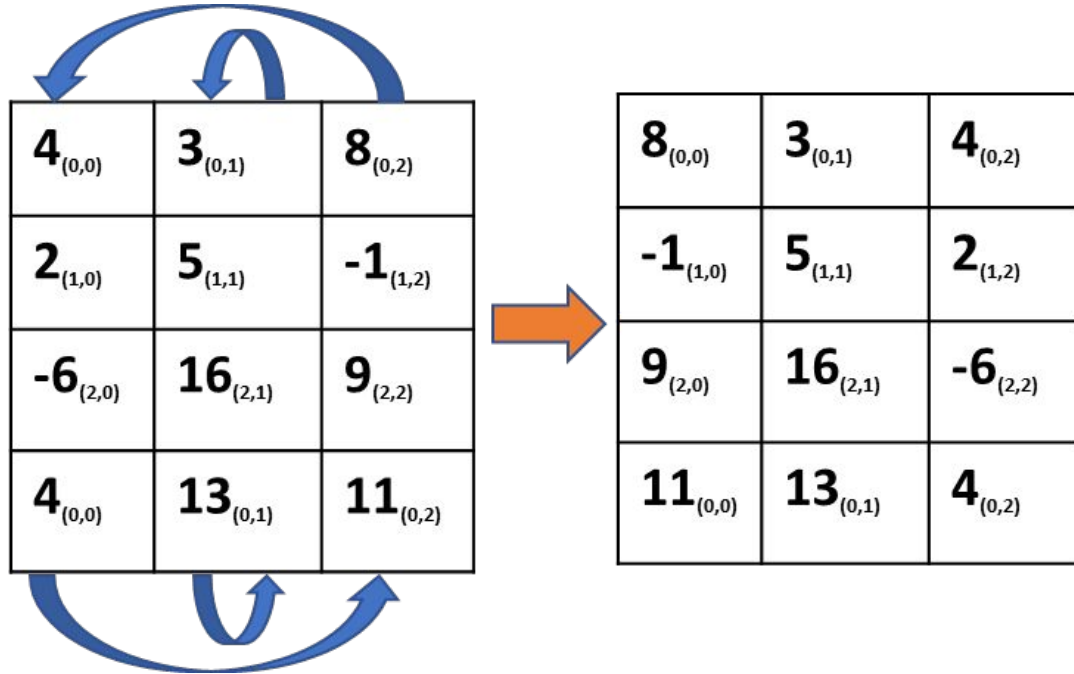
2D Array

Find The Smallest Number in Each Column in a 2D Array. Then Find The Largest Among Them

2D Array (Swap Columns of m*n matrix)



2D Array (Swap Columns of m*n matrix)



2D Array (Swap Columns of $m \times n$ matrix)

```
FUNCTION swap_columns(m: ARRAY)
  DECLARE row, col: INTEGER
  row, col = m.DIMENSIONS
  FOR i = 0 TO row-1
    FOR j = 0 TO col/2
      temp = m[i][j]
      m[i][j] = m[i][col-1-j]
      m[i][col-1-j] = temp
    END FOR
  END FOR
  RETURN m
END FUNCTION
```

2D Array (Swap Columns of $m \times n$ matrix)

```
def swap_columns(m):  
    row,col = m.shape  
    for i in range(row):  
        for j in range(col//2):  
            m[i][j],m[i][col-1-j] = m[i][col-1-j],m[i][j]  
    return m
```

2D Array

Swap Rows of $m \times n$ matrix

2D Array

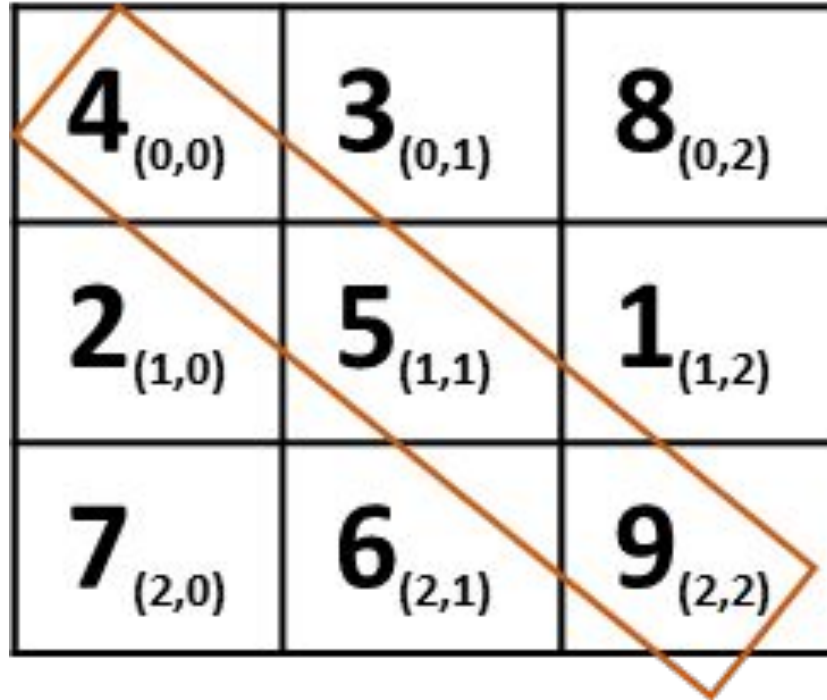
Row Wise Shift

Column Wise Shift

Row Wise Rotation

Column Wise Rotation

2D Array (Add elements of primary diagonal)



A 3x3 2D array is shown with its primary diagonal highlighted by an orange line. The elements and their indices are as follows:

4 $(0,0)$	3 $(0,1)$	8 $(0,2)$
2 $(1,0)$	5 $(1,1)$	1 $(1,2)$
7 $(2,0)$	6 $(2,1)$	9 $(2,2)$

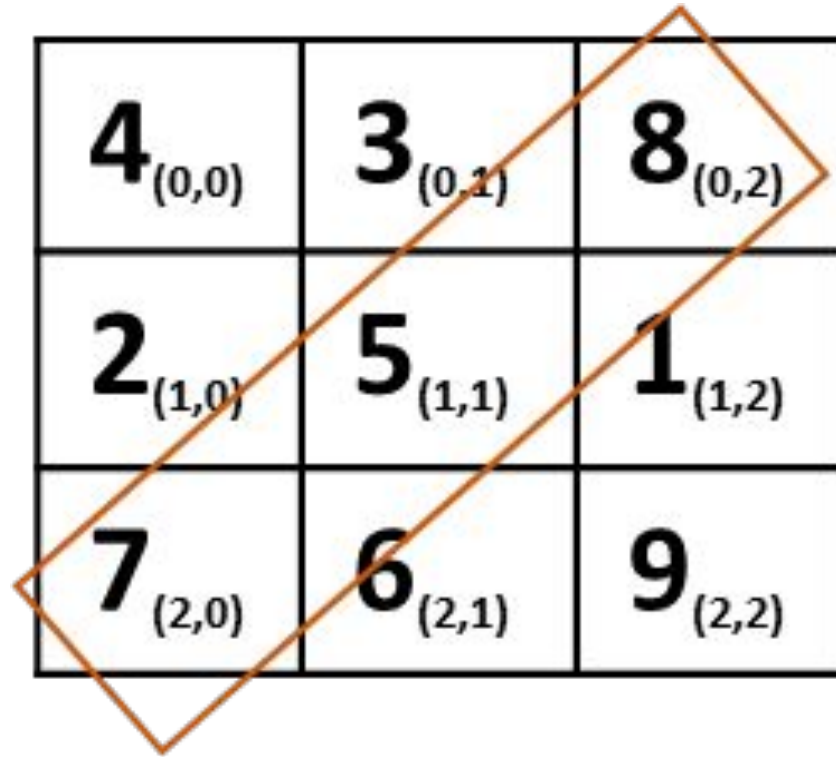
2D Array (Add elements of primary diagonal)

```
FUNCTION sum_primary_diagonal(m: ARRAY)
  DECLARE row, col: INTEGER
  row, col = m.DIMENSIONS
  ASSERT row = col, "Not a square matrix"
  DECLARE sum: INTEGER
  sum = 0
  FOR i = 0 TO row-1
    sum = sum + m[i][i]
  END FOR
  RETURN sum
END FUNCTION
```

2D Array (Add elements of primary diagonal)

```
def sum_primary_diagonal(m):  
    row,col = m.shape  
    assert (row==col), 'Not a square matrix'  
    sum = 0  
    for i in range(row):  
        sum += m[i][i]  
    return sum
```

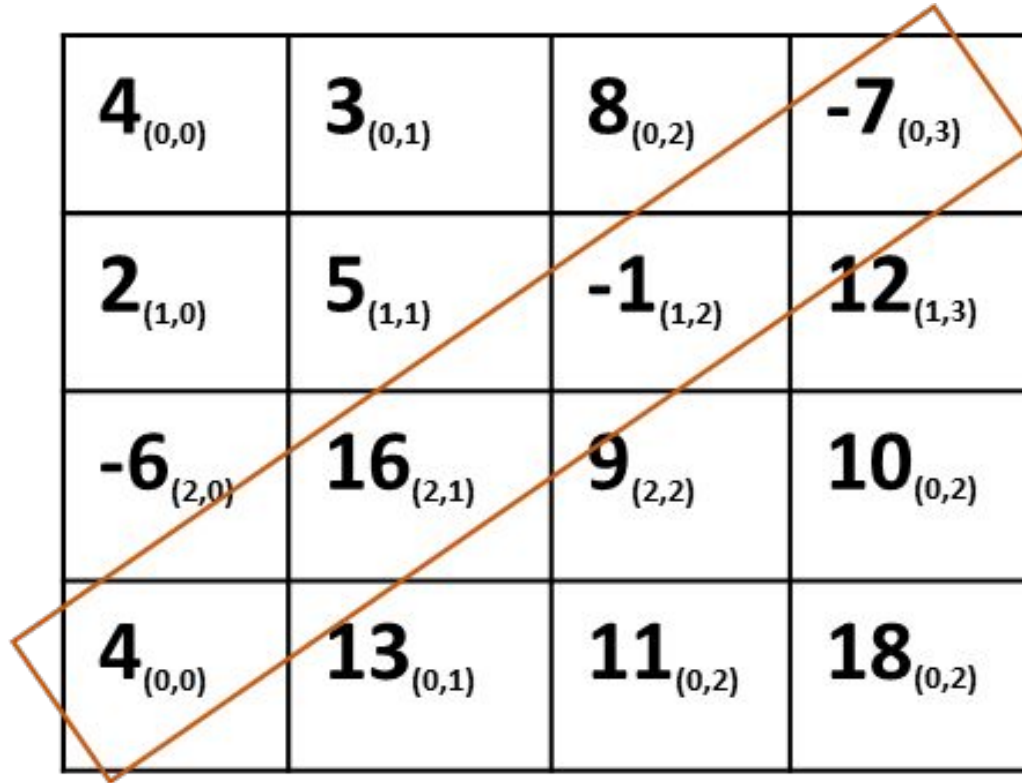
2D Array (Add elements of secondary diagonal)



A 3x3 2D array is shown with elements and their coordinates. The secondary diagonal, consisting of elements at (0,2), (1,1), and (2,0), is highlighted by an orange diamond shape.

4 (0,0)	3 (0,1)	8 (0,2)
2 (1,0)	5 (1,1)	1 (1,2)
7 (2,0)	6 (2,1)	9 (2,2)

2D Array (Add elements of secondary diagonal)



4 _(0,0)	3 _(0,1)	8 _(0,2)	-7 _(0,3)
2 _(1,0)	5 _(1,1)	-1 _(1,2)	12 _(1,3)
-6 _(2,0)	16 _(2,1)	9 _(2,2)	10 _(0,2)
4 _(0,0)	13 _(0,1)	11 _(0,2)	18 _(0,2)

2D Array (Add 2 Matrices)

```
FUNCTION add_matrix(m: ARRAY, n: ARRAY)
  DECLARE r_m, c_m, r_n, c_n: INTEGER
  r_m, c_m = m.DIMENSIONS
  r_n, c_n = n.DIMENSIONS
  ASSERT r_m = r_n AND c_m = c_n, "Dimension mismatch"
  DECLARE result: ARRAY [0:r_m-1, 0:c_m-1] OF INTEGER
  FOR i = 0 TO r_m-1
    FOR j = 0 TO c_m-1
      result[i][j] = m[i][j] + n[i][j]
    END FOR
  END FOR
  RETURN result
END FUNCTION
```

2D Array (Add 2 Matrices)

```
def add_matrix(m,n):  
    r_m, c_m = m.shape  
    r_n, c_n = n.shape  
    assert (r_m == r_n and c_m == c_n), 'Dimension mismatch'  
    result = np.zeros((r_m,c_m), dtype=int)  
    for i in range(r_m):  
        for j in range(c_m):  
            result[i][j] = m[i][j]+n[i][j]  
    return result
```

2D Array (Multiply Matrices)

```
matrix1 = [[12,7,3],  
            [4 ,5,6],  
            [7 ,8,9]]  
matrix2 = [[5,8,1],  
            [6,7,3],  
            [4,5,9]]
```

2D Array (Multiply Matrices)

```
FUNCTION multiply(m: ARRAY, n: ARRAY)
  DECLARE r_m, c_m, r_n, c_n: INTEGER
  r_m, c_m = m.DIMENSIONS
  r_n, c_n = n.DIMENSIONS
  ASSERT c_m = r_n, "Cannot multiply"
  DECLARE result: ARRAY [0:r_m-1, 0:c_n-1] OF INTEGER
  FOR i = 0 TO r_m-1
    FOR j = 0 TO c_n-1
      FOR k = 0 TO c_m-1
        result[i][j] = result[i][j] + m[i][k] * n[k][j]
      END FOR
    END FOR
  END FOR
  RETURN result
END FUNCTION
```

2D Array (Multiply Matrices)

```
def multiply(m,n):  
    r_m, c_m = m.shape  
    r_n, c_n = n.shape  
    assert c_m == r_n, 'Cannot multiply'  
    result = np.zeros((r_m,c_n), dtype=int)  
    for i in range(r_m):  
        for j in range(c_n):  
            for k in range(c_m):  
                result[i][j] += m[i][k]*n[k][j]  
    return result
```

MultiDimensional Array (Multiply Matrices)

Multiply Matrices (Geeks for Geeks)